

RELATÓRIO TÉCNICO: SISTEMA DE OTIMIZAÇÃO DE ROTAS

Disciplina: Programação Orientada a Objetos (POO)

Desenvolvedor: Arthur Vieira Corrêa

Linguagem: Python 3

1. Descrição do Problema

O sistema resolve o **Problema do Caixeiro Viajante com Restrição de Capacidade (CVRP)**. O objetivo é calcular a sequência ideal de entregas partindo de um depósito central (coordenada 0,0), reduzindo a quilometragem percorrida e garantindo que o peso total das mercadorias não ultrapasse a capacidade máxima de carga do veículo escolhido.

2. Modelagem Orientada a Objetos (POO)

O sistema foi dividido em módulos independentes (.py), utilizando as seguintes estruturas de classes:

- **Cliente:** Controla o cadastro do comprador (nome e identificação).
- **Ponto_entrega:** Armazena e expõe as coordenadas cartesianas X e Y do destino usando números inteiros (int).
- **Pedido:** Associa o pacote ao seu peso (kg). Aplica o conceito de **Composição**, contendo internamente um objeto de **Cliente** e um de **Ponto_entrega**.
- **Veiculo:** Guarda os dados físicos do automóvel (modelo e limite máximo de peso suportado).
- **Rota:** Centraliza o motorista, o veículo utilizado e gerencia uma lista dinâmica de objetos do tipo **Pedido**.
- **OtimizadorRotas:** O motor de cálculo do sistema. Isola toda a lógica matemática da interface com o usuário.

Pilares de POO Aplicados:

1. **Encapsulamento:** Os atributos começam com underline (ex: `self._pedidos`) para impedir alteração externa direta. O acesso é feito de forma segura apenas por propriedades `@property` (Getters).
2. **Polimorfismo:** Sobrescrita do método especial `__str__` em todas as classes para personalizar e formatar o texto exibido quando os objetos são impressos na tela (`print(objeto)`).

3. O Algoritmo do Vizinho Mais Próximo

A ordenação lógica das entregas funciona através dos seguintes passos automatizados:

1. **Origem:** O veículo inicia a viagem na coordenada originária do Depósito (0, 0).
2. **Cálculo Geométrico:** O sistema calcula a distância euclidiana entre a posição atual e todos os clientes restantes usando a fórmula da hipotenusa:
$$d = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$
3. **Seleção:** O cliente geograficamente mais próximo é escolhido.
4. **Checagem de Peso:** O sistema valida se o peso desse pedido cabe no veículo. Se couber, o pedido é fixado no roteiro e a coordenada do cliente passa a ser a nova posição atual do caminhão. Se não couber, o pedido vai para a lista de "Não Alocados".
5. **Ciclo e Retorno:** O pedido avaliado é retirado da fila de processamento (evitando loops infinitos). O processo se repete até varrer todos os pacotes e, no final, soma-se a distância de volta ao depósito.

4. Resultado de Execução (Cenário de Teste)

Durante a simulação utilizando um veículo Fiat Uno (capacidade de 200 kg) e pedidos que totalizavam 320 kg, o sistema barrou o excesso de carga e gerou a seguinte saída exata no console:

Plaintext

===== ROTA OTIMIZADA =====

Veículo: Fiat Uno

Motorista: Arthur

Capacidade Máxima: 200.0 kg

Carga Total Alocada: 180.00 kg

Distância Total do Circuito: 10.99 km

Itinerário Planejado:

Depósito -> P001 -> 0 -> Depósito

Pedidos Não Alocados (Capacidade Excedida):

-> P003 - Peso: 60.0 kg

-> P002 - Peso: 50.0 kg

=====

5. Conclusão

O projeto atingiu todos os requisitos obrigatórios. A arquitetura em POO garantiu segurança no tráfego de dados internos através de métodos de encapsulamento, e o algoritmo de roteamento cumpriu seu papel de forma estável, oferecendo uma solução funcional e livre de travamentos para o problema proposto.